Python | 0

(Task 1 of 11) In this tutorial, we will learn about **variable assignments** and **mutable variables**.

The following program illustrates the new concepts.

Python [Run ▶] Pseudo

```
x = 2          let x = 2
print(x)       print(x)
x = x + 1      x = x + 1
print(x)       print(x)
x = x * 2      x = x * 2
print(x)       print(x)
```

This program produces `2 3 6`. It first defines `x` and binds `x` to `2`. The first variable assignment `x = x + 1` **mutates** (the binding of) `x`. After that, `x` is bound to the value of `x + 1`, which is `3`. The next variable assignment again mutates `x` and binds `x` to the value of `x * 2`; this uses the new value of `x`, which is `3`, resulting in `3 * 2`, which is `6`.

---

Python | 1

(Task 2 of 11) What is the result of running this program?

Python [Run ▶] Scala 3

```
rent = 10          var rent = 10
rent = 10 * 2      rent = 10 * 2
print(rent)        println(rent)
```

2

```
20
```

3

You predicted the output correctly 🎉🎉🎉

`rent` is bound to `10` initially. The variable assignment mutates `rent` to `20`. So, when we print the value of `rent` after the variable assignment, we see `20`.

Click here to run this program in the Stacker.

---

Python | 4

(Task 3 of 11) What is the result of running this program?

Python [Run ▶] Scala 3

```
x = [ 55 ]              var x = Buffer(55)
v = [ x, 55, 55 ]       var v = Buffer(x, 55, 55)
x = [ 66 ]              x = Buffer(66)
print(v)                println(v)
```

`[[66], 55, 55]` 5

The answer is `[[55], 55, 55]`. 6

Textual explanation

You might think the `0`-th and first element of `v` refers to `x`. However, arrays refer to values, so it actually refers to the one-element array, i.e., *the value of* `x`. In SMoL, variable assignments change *only* the mutated variables.

Click here to run this program in the Stacker.

Graphical explanation

---

Python | 7

What is the result of running this program?

Python [Run ▶] JavaScript

```
x = [ 0 ]          let x = [ 0 ];
v = [ 2, x, 3 ]    let v = [ 2, x, 3 ];
x = [ 1 ]          x = [ 1 ];
print(v)           console.log(v);
```

`[2, [0], 3]` 8

You predicted the output correctly 🎉🎉🎉 9

---

Python | 10

(Task 4 of 11) What is the result of running this program?

Python [Run ▶] JavaScript

```
x = 12             let x = 12;
def f(y):          function f(y) {
    y = 0            y = 0;
    return x         return x;
print(f(x))        }
                   console.log(f(x));
```

⚠️ Scala 3 behaves differently.

`12` 11

You predicted the output correctly 🎉🎉🎉 12

The function call binds `y` to `12`. The variable assignment mutates the value of `y` to `0`, but `x` is still bound to `12`.

Click here to run this program in the Stacker.

---

Python |

(Task 5 of 11) What is the result of running this program?

Python [Run ▶]

```
m = 40
n = m
n = 22
print(m)
print(n)
```

Pseudo

```
let m = 40
let n = m
n = 22
print(m)
print(n)
```

```
40 22
```

You predicted the output correctly 🎉🎉🎉

`m` and `n` are bound to `40`. The variable assignment mutates the binding of `n`. So, `n` is eventually bound to `22`.

Click here to run this program in the Stacker.

---

Python |

(Task 6 of 11) What is the result of running this program?

Python [Run ▶]

```
x = 59
v = [ 68, 57, x ]
x = 74
print(v)
```

Pseudo

```
let x = 59
let v = vec[68, 57, x]
x = 74
print(v)
```

```
[68, 57, 59]
```

You predicted the output correctly 🎉🎉🎉

`v` is bound to the array created by `[ 68, 57, x ]`. `[ 68, 57, x ]` is a function call. `x` is evaluated at the point of evaluating `[ 68, 57, x ]`. That is why the array's content is `68`, `57`, and `59`. The subsequent variable assignment mutates `x`. But this doesn't impact the array because the array refers the value `59` rather than `x`.

Click here to run this program in the Stacker.

---

Python |

(Task 7 of 11) What is the result of running this program?

Python [Run ▶] JavaScript

```
x = 12              let x = 12;
def f(y):           function f(y) {
    global x          x = 0;
    x = 0             return y;
    return y        }
print(f(x))         console.log(f(x));
```

0

20

21

The answer is 12.

Textual explanation

You might think the function call f(x) binds y to x, so changing x will change y. However, we learned in previous tutorials that variables are bound to values, so y is bound to 12, i.e., *the value of* x. In SMoL, variable assignments change *only* the mutated variables.

Click here to run this program in the Stacker.

---

Python |

What is the result of running this program?

Python [Run ▶] Scala 3

```
a = 1                   var a = 1
def foobar(b):          def foobar(b : Int) =
    global a              a = 2
    a = 2                 b
    return b            println(foobar(a))
print(foobar(a))
```

1

23

You predicted the output correctly 🎉🎉🎉

24

---

Python |

(Task 8 of 11) What is the result of running this program? 25

Python [Run ▶] Pseudo

```
x = 80      let x = 80
y = x       let y = x
x = 32      x = 32
print(x)    print(x)
print(y)    print(y)
```

```
32 80
```
26

You predicted the output correctly 🎉🎉🎉 27

The first definition binds `x` to `80`. The second definition binds `y` to the value of `x`, which is `80`. The variable assignment mutates the binding of `x`, so `x` is now bound to `32`. But `y` is still bound to `80`.

Click here to run this program in the Stacker.

(Task 9 of 11) What did you learn about variable assignment from these programs? 28

they are very important 29

(Task 10 of 11) Variable assignments change *only* the mutated variables. That is, variables are not aliased. 30

(Note: some programming languages (e.g., C++ and Rust) allow variables to be aliased. However, even in those languages, variables are not aliased by default.)

Any feedback regarding these statements? Feel free to skip this question. 31

*(You skipped the question.)* 32

(Task 11 of 11) Please scroll back and select 1-3 programs that make the above point. 33

You don't need to select *all* such programs.

(*You selected 3 programs*) 34

Okay. How do these programs (19,22,25) support the point? 35

boom 36

Let's review what we have learned in this tutorial. 37

---

Variable assignments change *only* the mutated variables. That is, variables are not aliased.

(Note: some programming languages (e.g., C++ and Rust) allow variables to be aliased. However, even in those languages, variables are not aliased by default.)

---

You have finished this tutorial 🎉🎉🎉

Please print the finished tutorial to a PDF file so you can review the content in the future. **Your instructor (if any) might require you to submit the PDF.**

Start time: 1762803232294